PROJETO DE SOFTWARE · ENTREGA FINAL

Encurtador de URLs em C++

Implementação HTTP com persistência e analytics

Autor: Douglas Ramos Jordão
RA: 2245260101
Repositório: github.com/douglass-blip/BOOTCAMPIII-ENTREGA1
Execução individual: sem revisão por pares declarada
Data do relatório: 29 de outubro de 2026

1. Resumo da entrega

O projeto implementa um serviço de encurtamento de URLs em C++17. A aplicação expõe endpoints HTTP para criar links, redirecionar e consultar métricas. Os registros e eventos de clique podem ser persistidos em arquivos JSON Lines, e os principais componentes têm testes unitários e de integração.

37/37
testes CTest aprovados localmente

85% / 86%
coverage de linhas local / CI (escopo src/)

PR #27
integrado em develop

PR #28
promoção para main aguarda aprovação humana

Escopo funcional

- `POST /shorten`: cria código curto para URL válida e aceita expiração opcional.
- `GET /{short_code}`: redireciona com HTTP 302; códigos inexistentes ou expirados retornam 404.
- `GET /{short_code}/stats`: apresenta destino, cliques e timestamps associados.
- Geração base62, validação de URL e código, resolução de colisões e persistência local.

Estado de publicação: o commit de merge do PR #27 em `develop` é `517f48c2985f53a674e866fca0967d164fa80a56`. O PR #28 está aberto; não houve merge em `main`, tag ou release porque a regra do repositório exige uma aprovação humana.

2. Arquitetura e decisões

A API HTTP, implementada com cpp-httplib, delega a lógica de negócio a `RedirectService`. Esse serviço coordena a geração de códigos, o armazenamento de URLs e o rastreamento de cliques. `UrlStore` e `ClickTracker` têm implementações adequadas a testes em memória e persistência JSON Lines para execução local.

API HTTP. Rotas, validação dos payloads e respostas HTTP/JSON; testes exercitam criação, redirect, stats e entradas inválidas.

RedirectService. Orquestra criação, expiração, destino e contabilização; links expirados não recebem novos cliques.

CodeGenerator. Gera códigos base62 e consulta colisões; `std::hash` não garante códigos estáveis entre toolchains.

UrlStore / ClickTracker. Armazenam links e histórico de cliques; arquivos locais são sincronizados dentro de um processo, e compartilhar entre processos não é suportado.

ADRs consolidados

1. **Geração de códigos:** hash truncado com salt de tentativa e resolução de colisão pelo storage; sem promessa de estabilidade entre implementações de `std::hash`.
2. **Persistência:** append-only em JSON Lines para simplicidade e recuperação após reinício; linha inválida causa erro explícito, não descarte silencioso.
3. **Dependências:** cpp-httplib, nlohmann/json e GoogleTest obtidos por CMake FetchContent em versões fixadas.
4. **Expiração:** link expira quando `now >= expires_at`; analytics permanece disponível, mas redirect expirado retorna 404 sem incrementar cliques.

3. Verificação e resultados

Build local — passou. CMake 4.4.4, MinGW-w64 GCC 16.2.0.

CTest local — 37/37 aprovados. Suíte inclui API, persistência, concorrência, expiração e datas.

Execução embaralhada — 5 execuções aprovadas. 37 testes por execução com `--gtest_shuffle`.

Coverage local — 507/590 linhas (85%). GCC instrumentado e gcovr 8.4; limite automatizado de 70%.

Coverage CI — 501/579 linhas (86%). Artefato da execução 40872149947.

CI — build/teste e coverage passaram. PR #28 no commit `92a548a7c7da22ad5ce9fc0357e1ee5435b9b34a`; execução 40872149947.

Docker Compose — configuração válida. `docker compose --profile test config --quiet` passou; containers não iniciados porque o daemon Docker estava indisponível.

O workflow atual usa AddressSanitizer e UndefinedBehaviorSanitizer no job de build/teste. Não há evidência de ThreadSanitizer nesta validação do CI. Cobertura e testes medem caminhos exercitados; não provam ausência de defeitos ou atendimento exaustivo a todos os requisitos.

4. Histórico, governança e estado GitHub

PR #27 — implementação e finalização. Mesclado em `develop` em 29/10/2026; merge commit `517f48c2985f53a674e866fca0967d164fa80a56`.

PR #28 — promoção develop → main. Aberto, checks verdes, mas bloqueado por `REVIEW_REQUIRED`. A regra exige aprovação humana; nenhuma revisão foi fabricada e nenhum bypass foi usado.

Release. Nenhuma tag ou release foi publicada; depende da aprovação e do merge normal em `main`.

O autor informou que fará a entrega sozinho. As integrações anteriores via pull requests são histórico do repositório; este relatório não afirma revisão independente de colega nem aprovação humana no PR #28.

5. Retrospectiva do autor

Relato fornecido pelo autor sobre o ciclo de desenvolvimento: a dificuldade principal foi manter build e testes consistentes entre máquinas com/sem Docker e entre CMD/PowerShell, além de uma credencial Git de outra pessoa salva no Windows. Entre os problemas mencionados estão um `code_generator.cpp` antigo copiado por engano (237 bytes em vez de 1537), omissão de `url_store.cpp` no `CMakeLists.txt` e push 403 porque outra conta estava autenticada.

Segundo o relato, as respostas adotadas durante o ciclo foram tratar GitHub Actions como fonte de verdade para builds limpos no Ubuntu com ASan, UBSan e ThreadSanitizer; criar arquivos por comando e conferir tamanho/conteúdo antes de commits; trabalhar em clones limpos; configurar identidade Git apenas no repositório; e remover a credencial do Windows ao terminar. O aprendizado apontado pelo autor é exigir evidência objetiva (tamanho do arquivo, log do CI ou captura) e versionar a configuração do ambiente, em vez de depender da memória.

Esta seção é retrospectiva fornecida pelo autor sobre o ciclo; não afirma que todas as medidas foram executadas nesta sessão. O workflow atual comprovado neste relatório usa ASan e UBSan; ThreadSanitizer não foi alegado como resultado atual.

6. Uso de assistentes de IA

O autor confirmou uso de **Claude Code** e **GitHub Copilot**. Os registros históricos atribuem a Claude apoio inicial à especificação, decomposição e implementação. A finalização registrada nesta entrega usou GitHub Copilot para apoiar implementação, depuração, testes e documentação.

O relatório intermediário incluía uma comparação com Cursor, mas essa ferramenta não foi mantida nesta descrição: a confirmação posterior do autor identifica Claude Code e Copilot. Os registros permitem apenas uma comparação descritiva de etapas/contexto. Não houve experimento controlado, prompts pareados ou medição de tempo, defeitos ou cobertura por ferramenta; portanto, não há conclusão de superioridade entre elas. A implementação e o CI não permitem atribuir causalmente cada resultado a um assistente.

7. Análise ético-técnica

A. Alucinação e confiabilidade

Assistentes podem sugerir código ou explicações incorretos, inclusive em concorrência, datas e validação. Testes, contratos explícitos, sanitizers ASan/UBSan na CI e coverage são controles observáveis, mas não substituem compreensão e verificação do autor nem comprovam ausência de falhas.

B. Privacidade e dados

URLs e timestamps de cliques podem conter ou revelar dados pessoais. O serviço os persiste localmente em JSON Lines. Recomenda-se usar dados sintéticos em demonstrações, restringir acesso ao diretório e evitar publicar registros. Não há evidência suficiente para afirmar quais dados foram enviados às ferramentas de IA; nenhuma afirmação de ausência de compartilhamento é feita.

C. Propriedade intelectual

Dependências externas são obtidas por FetchContent em versões fixadas. Isso não equivale a uma auditoria de licenças. A redistribuição deve observar os avisos/licenças das versões usadas e as regras do curso para código assistido por IA; este relatório não emite conclusão jurídica nem inventa proveniência de trechos.

D. Responsabilidade e validação humana

O trabalho é individual e não houve revisão por pares. CI e testes são controles automatizados, não aprovação de colega. O autor permanece responsável por compreender os resultados. A regra de aprovação humana no PR #28 é respeitada: enquanto não houver aprovação, não há merge em `main` nem release.

8. Itens ainda dependentes

- Aprovação humana do PR #28 e merge normal para `main`; a release só poderá ocorrer depois disso.
- Experimento controlado entre Claude Code e GitHub Copilot, inclusive comparação com/sem `AGENTS.md`, caso exigido pela rubrica. Não foi inventado resultado.
- Template e critérios oficiais da Entrega 2 não foram fornecidos; este documento segue a estrutura e os dados de identificação do relatório intermediário fornecido como referência.
- Declarações pessoais sobre dados enviados, revisão de sugestões e propriedade intelectual devem ser completadas pelo autor se exigidas pelo curso.

Relatório preparado com base no estado verificável do repositório e em informações fornecidas pelo autor. O PDF intermediário foi usado como referência estrutural/identificatória; afirmações conflitantes ou não verificadas foram omitidas ou qualificadas.